



郑州大学
ZHENGZHOU UNIVERSITY

PulseOS 决赛设计文档

郑州大学

石家誉 孔梦琪

2026 年 8 月

目录

第 1 章	测试问题与解决	1
1.1	跑通测试遇到的问题	1
1.2	测试计时	1
第 2 章	qperf 改造与分析	2
2.1	改造目标	2
2.2	分析结论	2
第 3 章	性能分析与优化	3
3.1	性能瓶颈及分析过程	3
3.2	优化方案	3
3.3	优化前后的效果对比	4
3.4	SMP 核数对比	5
第 4 章	AI 使用说明	6
4.1	使用定位	6
4.2	使用方式	6
4.3	验证与证据边界	6
4.4	AI 开发案例	6

第 1 章 测试问题与解决

1.1 跑通测试遇到的问题

跑通测试的过程中遇到了以下问题：

1. 旧 `PipeObject` 仅处理 `FIONREAD`，对未知请求返回 `ENOTTY`：为 `pipe` 增加 `FIONBIO` 支持。
2. `TTY ioctl` 的兼容回退被错误地应用到 `pipe` 等非终端对象：将兼容回退限制为真实的 `devfs TTY`，其他文件描述符返回 `ENOTTY`。
3. `execve/execveat` 会对多线程执行直接返回 `EAGAIN`：将整个 `exec` 串行化，并在不可逆切换前完成新映像装载，随后请求并等待 `sibling` 退出，再替换映像。
4. 将不含 `PT_INTERP` 的 `direct ET_DYN` 误判为不受支持：增加对 `direct ET_DYN` 的支持，并补齐 `load bias`、`auxv` 与初始栈。
5. 动态装载窗口约 62 MiB，无法容纳约 147 MiB 的 `PT_LOAD` 段，导致 `rust-ldd` 映射时报 `ENOMEM`：在共享动态区域按 `p_align` 搜索不重叠的空闲区进行装载。

1.2 测试计时

测试评分依据是内核提供的 `/proc/uptime`，因此我们为 PulseOS 实现了 `/proc/uptime` 这一评分时间来源。

第 2 章 qperf 改造与分析

2.1 改造目标

BuildStorm 持续时间长、并发任务多，对内核的内存、文件系统等各个模块都存在要求。对于原版 qperf，既没有 SMP 采样分析的能力，也无法把 futex、块设备等待和调度迁移等阻塞事件与任务对应，面对 buildstorm 这种复杂测试已经不足以使用了。因此我们对 qperf 进行了以下两部分改造：

1. 修改内核导出 ABI `__pulse_qperf_trace_v1`。QEMU 会匹配 ELF 解析其地址并传给 QEMU plugin，plugin 在该地址读取 ABI 参数并还原 trace 事件。
2. 在任务、调度、运行队列和等待路径记录任务元数据、调度切换、阻塞、唤醒、入队和退出事件。入队事件保留 spawn、wake、preempt、yield、迁移、deferred wake 和在启用对应路径时的 work-steal 原因；阻塞事件保留 wait reason、资源标识和唤醒来源。

该设计把工作负载阶段、CPU 路径与调度事件关联起来，用于定位问题；它只承担机制分析，不改变普通内核的正式构建结果。

2.2 分析结论

改造后的 qperf 对 PulseOS 的性能优化存在以下几点价值：

1. 我们可以通过采样结果找出下一步优化的方向，无论是 on-cpu 的性能问题，还是 off-cpu 的阻塞问题。
2. 我们还可以通过 qperf 进行修改前后的采样，以此来证明修改前后是否有效，目标路径开销是否降低。

第 3 章 性能分析与优化

3.1 性能瓶颈及分析过程

1. 热点定位：通过分析 qperf 采样数据，候选热点指向 file page fault、page-cache 和 copy 路径。
2. 冷 file-backed mmap 缺页：
 - 瓶颈：顺序首次访问会反复进入单页 file fault 和页缓存装载路径，重复的页查找、I/O 准备和发布工作成为首扫开销。
 - 分析：bw_mmap_rd 会预热、lat_mmap 主要测映射管理、iozone 本轮走 buffered read / pread，都不覆盖目标路径；因此构造独立的静态非 PIE 测试，首次顺序触碰未访问的 48 MiB 文件。
3. 并发 Ext4 持久写：
 - 瓶颈：多 worker 同时写不同文件时，共享 inode/metadata、目录 registry、block cache stripe、dirty owner 和 I/O range lock 产生竞争；fsync 与 close 又把 writeback 和元数据提交纳入关键路径。
 - 分析：用 8 个 worker、8 个文件、64 KiB 对齐请求、每 worker 44 MiB，并以 -e 计入 fsync、-c 计入 close；按 B-C-C-B 顺序比较 iozone Parent aggregate throughput，且将写操作与读操作分开判读。
4. 顺序 mmap 预取与匿名 fault 探针：
 - 瓶颈假设：预取可能减少 demand fault 等待，但也可能引入无用 I/O、缓存淘汰和锁等待；匿名 fault 的重复页表读锁查询则是局部可疑开销。
 - 分析：对预取使用 marker 界定的完整 BuildStorm A/B；对匿名 fault 先做源码和 qperf 机制检查。只有完成同一正式阶段的普通内核对照，才纳入端到端效果结论。

3.2 优化方案

1. 连续 file-backed fault-around：在确认访问连续且页偏移对齐后，将单页 ensure_page_resident 扩展为最多四页的批量预取和页缓存扩展；仍沿用 single-flight、generation 校验、写锁、逐页映射和本地 TLB flush，避免以正确性换取速度。

2. Ext4 分片与 range lock: 将 inode state/metadata cache 和目录 registry 分为 32 个 shard; 分散 block-cache stripe 与 dirty owner, 并设置 128 个 I/O range-lock bucket, 使不重叠范围的并发写不必争用同一把全局锁。
3. 匿名 fault PTE 查询去重: 在同一页表子树内复用 PageTableReadGuard, 最多覆盖连续四页, 减少重复查询; 分配、PTE 写锁、映射和 TLB 刷新路径保持不变。该方案目前只有源码支持, 尚无匹配的完整 A/B。
4. 顺序 mmap 预取: 采用每文件 single-flight、全局 两个预取槽位和 try-lock admission, 忙时优先 demand fault; 设计上限制投机 I/O, 但必须以完整 BuildStorm 结果决定是否保留为优化。

3.3 优化前后的效果对比

1. 冷 file-backed mmap 首扫: 基线版本的四次中位数为 3.526777 s, 候选版本为 1.465404 s; 耗时减少 58.45%, 速度为 2.407x, 吞吐由 13.610 MiB/s 提升至 32.755 MiB/s。该结果支持四页 fault-around 在目标首扫路径有效, 但仍是 QEMU/guest 页缓存冷启动, 不等于物理盘冷读。
2. Ext4 并发持久写: 分片版本相对未分片基线的 Parent aggregate throughput 变化如下:
 - durable initial write: 91,741.29 -> 292,501.34 KiB/s, +218.83%;
 - durable rewrite: 101,993.00 -> 333,791.98 KiB/s, +227.27%;
 - durable pwrite: 99,947.95 -> 351,533.27 KiB/s, +251.72%;
 - mixed initial write: 94,002.81 -> 376,367.91 KiB/s, +300.38%;
 - mixed rewrite: 97,744.82 -> 313,203.82 KiB/s, +220.43%;
 - mixed random write: 69,207.76 -> 157,889.76 KiB/s, +128.14%。
 - 写入结果支持分片和 range lock 针对并发 fsync/close-inclusive 写路径有效; 不能外推为所有 Ext4 操作或物理盘吞吐全面提升。
3. Ext4 读路径: 读项单独列出, 避免把不同 I/O 语义合并为一个总分:
 - durable read: 248,443.80 -> 231,654.52 KiB/s, -6.76%;
 - durable re-read: 248,901.36 -> 251,009.16 KiB/s, +0.85%;
 - durable pread: 244,552.77 -> 247,864.29 KiB/s, +1.35%;
 - mixed reverse read: 138,645.27 -> 140,089.04 KiB/s, +1.04%;
 - mixed stride read: 229,588.76 -> 239,204.14 KiB/s, +4.19%;
 - mixed random read: 137,121.44 -> 141,853.52 KiB/s, +3.45%。

因此读路径没有出现与写路径同等级的稳定收益。

3.4 SMP 核数对比

在相同普通内核、镜像和 BuildStorm 输入下，仅改变 QEMU 运行期 vCPU 数。正式耗时取 guest BUILDSTORM_BEGIN 至 BUILDSTORM_COMPILE 的区间，并以单核结果计算加速比与 并行效率：

vCPU	正式耗时	相对单核加速	并行效率
1	2883.84 s	1.000x	100.0%
2	1588.21 s	1.816x	90.8%
4	1109.79 s	2.599x	65.0%
8	949.03 s	3.039x	38.0%

核数从 1 增至 8 时，正式耗时持续下降：2 核、4 核和 8 核相对单核分别缩短 44.9%、61.5% 和 67.1%；即使从 4 核增加到 8 核仍获得 1.169 倍加速。数据表明该 BuildStorm 负载能够利用多个 CPU，PulseOS 的 SMP 并行是有效的；并行效率随核数增加 而下降，则说明收益并非线性扩展，PulseOS 仍然存在改进空间。

第 4 章 AI 使用说明

4.1 使用定位

在决赛线上阶段，AI 的主要用途是分析 `qperf` 采样、测试日志和相关源码路径，梳理可能的性能热点、阻塞关系与后续排查方向，以及选择或编写合适的测试对修改前后的效果进行验证与测试。同时，AI 也用于对部分性能热点路径进行修改优化。

4.2 使用方式

每次协作开始前，我们都会先提供明确的任务范围，例如相关源码路径、复现步骤、构建或运行日志、采样产物和预期行为。AI 的输出均视为待核查的建议，而不直接采纳，主要包括以下几个方面：

1. 将日志、调用栈和采样结果关联到候选代码路径；
2. 对比修改前后的实现，提出兼容性、并发性等风险；
3. 协助拆分修改步骤，使结果更易于人工检查；
4. 协助整理测试记录、实验说明和设计文档。

对于涉及内存管理、调度、文件系统、网络 and 系统调用等内核关键路径的建议，我们会结合实际调用链以及 `qperf` 采样数据分析进行复核。

4.3 验证与证据边界

AI 可以帮助提出性能假设，但不能据此直接证明优化有效。`qperf` 专用内核提供的是 `on-CPU` 采样、调度与阻塞事件等机制层证据，用于定位候选热点。

性能结论须由人工复核后的源码分析与受控实验共同支持。对比测试需要固定内核与 ELF、插件与分析器、镜像、QEMU 参数及工作负载输入，并检查构建日志、运行日志和采样产物的完整性。普通内核的实际工作负载结果与 `qperf` 机制分析应分别记录，避免将 AI 推断、采样线索和可复现实验结果混为一谈。

4.4 AI 开发案例

以 `qperf` 的使用为例，我们会在完成采样后先交给 AI 做初步数据处理，并输出分析报告。随后结合上面的数据报告 and 实际调用链人工复核，再让 AI 协助给出修改方案和测试计划。方案确认后执行修改与测试，最后继续结合 AI 的分析结果验证优化是否有效。